```
function [grid_array_az,grid_array_r,Npsc,Npsp] =
ps_selection(filenames_slc,filenames_ifgs,filenames_h2ph,psc_selection_method,ps
c_selection_gridsize,psc_threshold,Npsc_selections,psp_selection_method,psp_thre
shold1,psp_threshold2,slc_selection,Btemp,ps_aoi,filename_water_mask,crop,crop_i
n,do_apriori_sidelobe_mask,calfactors,gamma_threshold,psc_distribution,livetime_
threshold,peak_tolerance,ref_cn)
% Function to read a stack of co-registered, calibrated slc's
% in buffers and to select persistent scatterer candidates (psc)
% using the amplitude dispersion (ferretti et al., 2001). Also
% potential ps are selected for latter use.
%
% Input:    - filenames_slc            filenames for calibrated slc's
%           - filenames_ifgs           filenames for interferograms
%           - filenames_h2ph           filenames for height to phase
%                                      factors
%           - psc_selection_method     psc selection method,
%                                      'threshold' or 'eachgrid'
%           - psc_selection_gridsize   gridsize [m]
%           - psc_threshold            amplitude dispersion threshold value
%           - Npsc_selections          number of psc selections
%           - psp_selection_method     psp selection method,
%                                      'highamp' or 'ampdisp'
%           - psp_threshold1           minimal number of ifgs above
%                                      psp_threshold2
%           - psp_threshold2           potential ps threshold [dB]
%           - slc_selection            selection of slc's used to select psc's
%           - Btemp                    temporal baselines [year]
%           - ps_aoi                   area of interest, 'filename',
%                                      [az0 azN r0 rN] or []
%           - filename_water_mask      mask for water areas, 'filename' or []
%           - crop                     borders of final crop
%           - crop_in                  borders of input crops
%           - do_apriori_sidelobe_mask do apriori sidelobe mask,
%                                      'yes', 'no' or 'notapply'
%           - calfactors               calibration factors
%
% Output:   - grid_array_az            array with borders gridcells in
%                                      azimuth direction
%           - grid_array_r             array with borders gridcells in
%                                      range direction
%           - Npsc                     number of psc's
%           - Npsp                     number of psp's
%
% ------------------------------------------------------------------------
% File............: ps_selection.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Authors.........: Freek van Leijen
%                   Gini Ketelaar
%                   Miguel Caro Cuenca
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
% ------------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log
%
% v1.7.2.8, Freek van Leijen
% - Calibration in separate function
% v1.7.2.8, Miguel Caro Cuenca
```

```
% - First order PSC selection based on spatial coherence (A. Hooper
% method)
% - Additional option to omit shifted grid (psc_distribution
% parameter)
% v1.7.2.13, Freek van Leijen
% - switch side_lobe_threshold turned back on
% v1.7.4.0, Freek van Leijen
% - psp validation output to file
% v1.7.5.3 -
% svn rev. 161: Piers Titus van der Torren
% - add simplified piers scatterer detection method as apriori sidelobe mask
% v1.7.7.4, Freek van Leijen
% - added inclusion of reference point in psc selection
%

% ---------------------------------------------------------------------
% Initialize
% ---------------------------------------------------------------------

ps_set_globals;

Nifgs = size(filenames_ifgs,1);
Nslc = size(filenames_slc,1);
Nslc_selected = length(slc_selection);
Npsc = zeros(Npsc_selections,1);
Npsp = zeros(Npsc_selections,1);

if isempty(ref_cn)
  ref_cn = [NaN NaN]; %only local, to enable check on coordinates
end


% locate potential ps, as implemented by DLR (see thesis Bert Kampes)
% pixel is potential ps if normalized RCS (sigma_naught) > psp_threshold2 (dB)
in minimal 65% of images
%
% The relation between amplitude and normalized RCS is
% A^2 = K*sigma_naught (approximately)
% For ERS2 SLCI processed at ESRIN from 20th of 1997, K = 93325.3 (linear scale)
% Therefore, A = sqrt(K*10^(psp_threshold2/10)) = 242.66 for -2dB
% for -4 dB, 192.75
% (See Laur et al., 2002 for more information)

K = 93325.3; % This value should be equal to the one used as
% reference during calibration
psp_amplitude = sqrt(K*10^(psp_threshold2/10));



% ---------------------------------------------------------------------
% Open the data files
% ---------------------------------------------------------------------

psc_temp_fid = repmat(NaN,Npsc_selections,1);
for z = 1:Npsc_selections
  psc_temp_fid(z) = fopen([project_id '_psc_temp_sel' num2str(z) '.raw'],'w'); %
file with temporary psc info
end

switch ps_eval_method
  case 'psp'
    psp_fid = repmat(NaN,Npsc_selections,1);
    for z = 1:Npsc_selections
      psp_fid(z) = fopen([project_id '_psp_2orig_sel' num2str(z) '.raw'],'w'); %
```

```
file with psp info
    end
    psp_validation_fid = fopen([project_id '_psp_validation.raw'],'w');
  case 'whole'
    Npsp = [];
  otherwise
    error('You specified a wrong ps_eval_method.');
end

mrm_fid = fopen([project_id '_mrm.raw'],'w');
% file with multi reflectivity map
amp_disp_fid = fopen([project_id '_amp_disp.raw'],'w');
% file with amplitude dispersion
side_lobe_fid = fopen([project_id '_side_lobe_mask.raw'],'w');
% file with side lobe mask

save_livetime = strcmp(do_apriori_sidelobe_mask,'piers');

if save_livetime
  livetime_fid = fopen([project_id '_livetime.raw'],'w');
  % file with livetime
end


% --------------------------------------------------------------------
% Determine buffersize
% --------------------------------------------------------------------

Nlines_g = round(psc_selection_gridsize/az_spacing);
Npixels_g = round(psc_selection_gridsize/r_spacing);

Ng_az = floor(Nlines/Nlines_g);
Ng_r = floor(Npixels/Npixels_g);
offset_az = floor(rem(Nlines,Nlines_g)/2);
offset_r = floor(rem(Npixels,Npixels_g)/2);

buffer_size_az = floor(max_mem_buffer/(8*8*Nifgs*Npixels*Nlines_g));
% buffer_size_az in number of gridcells in azimuth direction
% 6 because of 2 float array's, 2 complex array's, 1 multi image
% and 1 for the rest, 8 because of double

if (buffer_size_az == 0)
  buffer_size_az = 1;
  buffer_size_r = floor(max_mem_buffer/(8*8*Nifgs*Npixels_g*Nlines_g));
  if (buffer_size_r == 0)
    error('Not enough memory to proceed. Please increase the allocated memory or
reduce the grid size');
  end
  Nbuffers_r = floor(Ng_r/buffer_size_r);
  rem_buffer_r = rem(Ng_r,buffer_size_r);
else
  buffer_size_r = Ng_r;
  Nbuffers_r = 1;
  rem_buffer_r = 0;
end

if (buffer_size_az*Nlines_g > Nlines)
  buffer_size_az = Ng_az;
end

Nbuffers_az = floor(Ng_az/buffer_size_az);
rem_buffer_az = rem(Ng_az,buffer_size_az);
```

```
% ------------------------------------------------------------------------
% Determine borders gridcells
% ------------------------------------------------------------------------

grid_array_az = repmat(NaN,Ng_az,3);
grid_array_az(:,1) = [1 offset_az+Nlines_g+1:Nlines_g:offset_az+(Ng_az-
1)*Nlines_g+1]';
grid_array_az(:,2) = [offset_az+Nlines_g:Nlines_g:offset_az+ ...
                      (Ng_az-1)*Nlines_g Nlines]';
for v = 1:Nbuffers_az
  buffer = (v-1)*buffer_size_az+1:v*buffer_size_az;
  grid_array_az(buffer,3) = repmat(v,length(buffer),1);
end

if (rem_buffer_az ~= 0)
  buffer = v*buffer_size_az+1:Ng_az;
  grid_array_az(buffer,3) = repmat(v+1,length(buffer),1);
  Nbuffers_az = Nbuffers_az + 1; % add remaining buffer
end

grid_array_r = repmat(NaN,Ng_r,3);
grid_array_r(:,1) = [1 offset_r+Npixels_g+1:Npixels_g:offset_r+(Ng_r-
1)*Npixels_g+1]';
grid_array_r(:,2) = [offset_r+Npixels_g:Npixels_g:offset_r+ ...
                     (Ng_r-1)*Npixels_g Npixels]';

for v = 1:Nbuffers_r
  buffer = (v-1)*buffer_size_r+1:v*buffer_size_r;
  grid_array_r(buffer,3) = repmat(v,length(buffer),1);
end

if (rem_buffer_r ~= 0)
  buffer = v*buffer_size_r+1:Ng_r;
  grid_array_r(buffer,3) = repmat(v+1,length(buffer),1);
  Nbuffers_r = Nbuffers_r + 1; % add remaining buffer
end

% output to screen
fprintf(1,'\n');
fprintf(1,'Data devided in %3.0f buffers of %3.0f grid cells in azimuth
direction\n', ...
        Nbuffers_az,buffer_size_az);
if (rem_buffer_az ~= 0)
  fprintf(1,'plus a remaining buffer of %3.0f grid cells\n', ...
          rem_buffer_az);
end
fprintf(1,'and in %3.0f buffers of %3.0f grid cells in range direction\n', ...
        Nbuffers_r,buffer_size_r);
if (rem_buffer_az ~= 0)
  fprintf(1,'plus a remaining buffer of %3.0f grid cells\n', ...
          rem_buffer_r);
end
fprintf(1,'\n');



% ------------------------------------------------------------------------
% Main loop per buffer
% ------------------------------------------------------------------------

for v = 1:Nbuffers_az
```

```
  % ------------------------------------------------------------------------
  % Read data
  % ------------------------------------------------------------------------

  index_az = find(grid_array_az(:,3)==v);
  Ngrid_az = length(index_az);
  begin_buffer_az = grid_array_az(index_az(1),1);
  end_buffer_az = grid_array_az(index_az(end),2);
  buffer_size_az = end_buffer_az-begin_buffer_az+1;

  multi_refl_map = repmat(NaN,buffer_size_az,Npixels);
  amp_disp_map = repmat(NaN,buffer_size_az,Npixels);
  side_lobe_mask = repmat(NaN,buffer_size_az,Npixels);

  if save_livetime
      livetime = nan(buffer_size_az,Npixels);
  end

  for w = 1:Nbuffers_r

    fprintf(1,'Processing buffer %3.0f of %3.0f in azimuth and buffer %3.0f of
%3.0f in range\n',v,Nbuffers_az,w,Nbuffers_r)

    index_r = find(grid_array_r(:,3)==w);
    Ngrid_r = length(index_r);
    begin_buffer_r = grid_array_r(index_r(1),1);
    end_buffer_r = grid_array_r(index_r(end),2);
    buffer_size_r = end_buffer_r-begin_buffer_r+1;

    slc_array = repmat(NaN,[buffer_size_az buffer_size_r Nslc]);
    cpx_ifgs_array = repmat(NaN,[buffer_size_az buffer_size_r Nifgs]);
    ifgs_array = repmat(NaN,[buffer_size_az buffer_size_r Nifgs]);
    h2ph_array = repmat(NaN,[buffer_size_az buffer_size_r Nifgs]);

    for z = 1:Nifgs
      Nlines_file = crop_in(z,2)-crop_in(z,1)+1;
      loffset = crop(1)-crop_in(z,1);
      poffset = crop(3)-crop_in(z,3);
      slc_array(:,:,z) =
freadbk(char(filenames_slc(z)),Nlines_file,'cpxfloat32',...
                          begin_buffer_az+loffset,end_buffer_az+loffset,...
                          begin_buffer_r+poffset,end_buffer_r+poffset);
      disp(char(filenames_ifgs(z)))
      cpx_ifgs_array(:,:,z) =
freadbk(char(filenames_ifgs(z)),Nlines_file,'cpxfloat32',...

begin_buffer_az+loffset,end_buffer_az+loffset,...
                             begin_buffer_r+poffset,end_buffer_r+poffset);
      ifgs_array(:,:,z) = angle(cpx_ifgs_array(:,:,z));
      h2ph_array(:,:,z) =
freadbk(char(filenames_h2ph(z)),Nlines_file,'float32',...
                           begin_buffer_az+loffset,end_buffer_az+loffset,...
                           begin_buffer_r+poffset,end_buffer_r+poffset);
    end

    if Nslc == Nifgs+1
      % read last slc
      Nlines_file = crop_in(Nslc,2)-crop_in(Nslc,1)+1;
      loffset = crop(1)-crop_in(Nslc,1);
      poffset = crop(3)-crop_in(Nslc,3);

      fileinfo = dir(char(filenames_slc(Nslc)));
      Nel = (crop_in(Nslc,2)-crop_in(Nslc,1)+1)*(crop_in(Nslc,4)-
crop_in(Nslc,3)+1);
```

```
      if fileinfo.bytes==Nel*2*4
        master_format = 'cpxfloat32';
      elseif fileinfo.bytes==Nel*2*2;
        master_format = 'cpxint16';
      else
        error('Something is wrong with the file size of the master SLC.');
      end

      slc_array(:,:,Nslc) =
freadbk(char(filenames_slc(Nslc)),Nlines_file,master_format,...
                              begin_buffer_az+loffset,end_buffer_az+loffset,...
                              begin_buffer_r+poffset,end_buffer_r+poffset);
    end

    if ischar(ps_aoi)

      Nel_master = (crop_in(Nslc,2)-crop_in(Nslc,1)+1)*(crop_in(Nslc,4)-
crop_in(Nslc,3)+1);
      Nel_mrm = (crop(2)-crop(1)+1)*(crop(4)-crop(3)+1);

      fileinfo = dir(ps_aoi);

      if fileinfo.bytes==Nel_master
        Nlines_file = crop_in(Nslc,2)-crop_in(Nslc,1)+1;
        loffset = crop(1)-crop_in(Nslc,1);
        poffset = crop(3)-crop_in(Nslc,3);
      elseif fileinfo.bytes==Nel_mrm
        Nlines_file = crop(2)-crop(1)+1;
        loffset = 0;
        poffset = 0;
      end

      aoi_mask = freadbk(ps_aoi,Nlines_file,'uint8',...
                         begin_buffer_az+loffset,end_buffer_az+loffset,...
                         begin_buffer_r+poffset,end_buffer_r+poffset);
    end

    if ischar(filename_water_mask)

      Nel_master = (crop_in(Nslc,2)-crop_in(Nslc,1)+1)*(crop_in(Nslc,4)-
crop_in(Nslc,3)+1);
      Nel_mrm = (crop(2)-crop(1)+1)*(crop(4)-crop(3)+1);

      fileinfo = dir(filename_water_mask);

      if fileinfo.bytes==Nel_master
        Nlines_file = crop_in(Nslc,2)-crop_in(Nslc,1)+1;
        loffset = crop(1)-crop_in(Nslc,1);
        poffset = crop(3)-crop_in(Nslc,3);
      elseif fileinfo.bytes==Nel_mrm
        Nlines_file = crop(2)-crop(1)+1;
        loffset = 0;
        poffset = 0;
      end

      water_mask = freadbk(filename_water_mask,Nlines_file,'uint8',...
                           begin_buffer_az+loffset,end_buffer_az+loffset,...
                           begin_buffer_r+poffset,end_buffer_r+poffset);

      land_index = find(water_mask==0);
      if length(land_index)>100
        land_flag = 1; % skip selection if only water in buffer
      else
        land_flag = 0;
```

```
      end
    else
      land_flag = 1;
    end


    % ---------------------------------------------------------------------
    % Amplitude calibration
    % ---------------------------------------------------------------------

    amp_array = abs(slc_array);
    for k = 1:Nslc_selected
      amp_array(:,:,slc_selection(k)) =
amp_array(:,:,slc_selection(k))/calfactors(k);
    end
    mean_amp = nanmean(amp_array(:,:,slc_selection),3);
    multi_refl_map(:,begin_buffer_r:end_buffer_r) = mean_amp;

    if land_flag

      if ischar(filename_water_mask)
        [water_az,water_r] = find(water_mask);
        for k = 1:length(water_az)
          amp_array(water_az(k),water_r(k),:) = NaN; %added to mask water bodies
        end
        mean_amp = nanmean(amp_array(:,:,slc_selection),3);
      end

      std_amp = nanstd(amp_array(:,:,slc_selection),0,3);
      % the 0-flag causes std to normalize by (N-1)
      amp_disp = std_amp./mean_amp; % amplitude dispersion

      amp_disp_map(:,begin_buffer_r:end_buffer_r) = amp_disp;


      switch do_apriori_sidelobe_mask %amplitude based
        case {'yes','notapply'}

          %
----------------------------------------------------------------------
          % Create sidelobe mask
          %
----------------------------------------------------------------------

          switch psp_selection_method
            case 'highamp'
              side_lobe_threshold = 0.4;
            case 'ampdisp'
              side_lobe_threshold = psp_threshold1;
            otherwise
              side_lobe_threshold = inf;
          end
          side_lobe_threshold = inf;
          %side_lobe_threshold = 0.4;

          thr = 0.6;

          mean_amp(amp_disp>side_lobe_threshold) = 0;
          mean_amp(isnan(mean_amp)) = 0.001; %should be selected as side lobe,
therefore not 0 but small number (logical(0.001)=1;


          if ~isempty(find(mean_amp~=0))
```

```
            %sl_filt = [0 0 0;1 1 1;0 0 0];
            %loc_max=imregionalmax(mean_amp,sl_filt);
            loc_max=imregionalmax(mean_amp,8);
            [r c]=find(loc_max);

            % check dependence in range

            % check dependence until no dependency left in range direction
            [r_r ind]=sort(r);
            c_r=c(ind);

            r_r_out=repmat(NaN,size(r_r));
            c_r_out=repmat(NaN,size(c_r));

            r_check=unique(r_r);
            for k=1:numel(r_check)

              ind_c=find(r_r==r_check(k));
              mean_amp_row = mean_amp(r_check(k),c_r(ind_c));
              rm_index = logical(zeros(size(ind_c)));

              while sum(rm_index)~=length(rm_index)
                [dummy,max_ind] = nanmax(mean_amp_row);

                r_depend =
abs(sum(exp(i*(repmat(angle(slc_array(r_check(k),c_r(ind_c(max_ind)),1:Nifgs)),
[1 length(ind_c) 1]) ...
                        -
angle(slc_array(r_check(k),c_r(ind_c),1:Nifgs)))),3)/Nifgs);
                r_depend(rm_index) = NaN;

                r_r_out(ind_c(max_ind)) = r_r(ind_c(max_ind));
                c_r_out(ind_c(max_ind)) = c_r(ind_c(max_ind));

                coh = r_depend>thr;
                coh_ind = find(~coh);

                pos_ind = find(coh_ind-max_ind>0);
                if ~isempty(pos_ind)
                  [dummy,max_coh_ind] = min(pos_ind);
                  max_coh_ind = coh_ind(pos_ind(max_coh_ind))-1;
                else
                  max_coh_ind = numel(coh);
                end

                neg_ind = find(coh_ind-max_ind<0);
                if ~isempty(neg_ind)
                  [dummy,min_coh_ind] = max(neg_ind);
                  min_coh_ind = coh_ind(neg_ind(min_coh_ind))+1;
                else
                  min_coh_ind = 1;
                end

                mean_amp_row(min_coh_ind:max_coh_ind) = NaN;
                rm_index(min_coh_ind:max_coh_ind) = 1;

              end
            end
            r_r_out(ind_c) = r_r(ind_c);% remaining main lobe
            c_r_out(ind_c) = c_r(ind_c);

            r_r = r_r_out(~isnan(r_r_out));
            c_r = c_r_out(~isnan(c_r_out));
```

```
            % check dependence in azimuth

            % check dependence until no dependency left in azimuth direction
            [c_r ind]=sort(c_r);
            r_r=r_r(ind);

            r_r_out=repmat(NaN,size(r_r));
            c_r_out=repmat(NaN,size(c_r));

            c_check=unique(c_r);
            for k=1:numel(c_check)

              ind_r=find(c_r==c_check(k));
              mean_amp_col = mean_amp(r_r(ind_r),c_check(k));
              rm_index = logical(zeros(size(ind_r)));

              while sum(rm_index)~=length(rm_index)
              [dummy,max_ind] = nanmax(mean_amp_col);

              c_depend =
abs(sum(exp(i*(repmat(angle(slc_array(r_r(ind_r(max_ind)),c_check(k),1:Nifgs)),
[length(ind_r) 1 1]) ...
                                  -
angle(slc_array(r_r(ind_r),c_check(k),1:Nifgs)))),3)/Nifgs);
              c_depend(rm_index) = NaN;

              r_r_out(ind_r(max_ind)) = r_r(ind_r(max_ind));
              c_r_out(ind_r(max_ind)) = c_r(ind_r(max_ind));

              coh = c_depend>thr;
              coh_ind = find(~coh);

              pos_ind = find(coh_ind-max_ind>0);
              if ~isempty(pos_ind)
                [dummy,max_coh_ind] = min(pos_ind);
                max_coh_ind = coh_ind(pos_ind(max_coh_ind))-1;
              else
                max_coh_ind = numel(coh);
              end

              neg_ind = find(coh_ind-max_ind<0);
              if ~isempty(neg_ind)
                [dummy,min_coh_ind] = max(neg_ind);
                min_coh_ind = coh_ind(neg_ind(min_coh_ind))+1;
              else
                min_coh_ind = 1;
              end

              mean_amp_col(min_coh_ind:max_coh_ind) = NaN;
              rm_index(min_coh_ind:max_coh_ind) = 1;
              end
            end

            r_r_out(ind_r) = r_r(ind_r);% remaining main lobe
            c_r_out(ind_r) = c_r(ind_r);

            r_r = r_r_out(~isnan(r_r_out));
            c_r = c_r_out(~isnan(c_r_out));


            %%%%%temp solution from here
            r_r_out = r_r;
            c_r_out = c_r;
```

```
          % additional check in range and azimuth direction with nearby PS
          rmind=1;
          while and(numel(c_r_out)>1,numel(rmind)>0)
            rmind=[];
            for k=1:numel(c_r_out)
            % find closest PS
            dist=(c_r_out-c_r_out(k)).^2+(r_r_out-r_r_out(k)).^2;
            ind=find(dist==min(dist(dist>0)));ind=ind(1);
            %rc_depend=(abs(sum(exp(i*(ifgs_array(r_r_out(k),c_r_out(k),:)...
            %
-ifgs_array(r_r_out(ind),c_r_out(ind),:))))))/Nifgs;


rc_depend=(abs(sum(exp(i*(angle(slc_array(r_r_out(k),c_r_out(k),1:Nifgs))...

-angle(slc_array(r_r_out(ind),c_r_out(ind),1:Nifgs)))))))/Nifgs;
            if rc_depend>thr
              if
mean_amp(r_r_out(k),c_r_out(k))<mean_amp(r_r_out(ind),c_r_out(ind))
                rmind=[rmind;k];
              else
                rmind=[rmind;ind];
              end
            end
            end
            c_r_out(rmind)=[];
            r_r_out(rmind)=[];
          end

          r_r = r_r_out;
          c_r = c_r_out;
          %%%%%end temp solution

            % create side lobe mask
            sl_mask = logical(mean_amp);
            ind = sub2ind(size(sl_mask),r_r,c_r);
            sl_mask(ind) = 0;
            side_lobe_mask(:,begin_buffer_r:end_buffer_r) = sl_mask;

            switch do_apriori_sidelobe_mask %amplitude based
              case {'yes'}      % insert side lobe mask
                [sl_az,sl_r] = find(sl_mask);
                for k = 1:length(sl_az)
                  amp_array(sl_az(k),sl_r(k),:) = NaN;
                end
                amp_disp(sl_mask) = NaN;
                amp_disp_map(:,begin_buffer_r:end_buffer_r) = amp_disp;
                if ischar(ps_aoi)
                  aoi_mask(sl_mask) = 0;
                end
            end

          else
            sl_mask = logical(mean_amp);
          end

          side_lobe_mask(:,begin_buffer_r:end_buffer_r) = sl_mask;

        case 'piers'
          % threshold is percentage of slc's that has an amplitude peak
          %livetime_threshold = 0.2;

          % include local near maxima
          %peak_tolerance = 0.9;
```

```
          % slc based scatterer detection
          slccount = size(slc_array,3);

          magslcs = abs(slc_array);
          peaks=imdilate(magslcs,ones(3)).*peak_tolerance <= magslcs;
          lt=sum(peaks,3);

          %lt=sum(imregionalmax(abs(slc_array),8),3);

          m = imregionalmax(lt,8);

          sl_mask = m & (lt > slccount*livetime_threshold);


          side_lobe_mask(:,begin_buffer_r:end_buffer_r) = ~sl_mask; % invert to
make mask

          livetime(:,begin_buffer_r:end_buffer_r) = lt;

        case 'no'
          sl_mask = zeros(size(amp_disp));
          side_lobe_mask(:,begin_buffer_r:end_buffer_r) = sl_mask;

        otherwise
          error('You should specify ''yes'', ''no'' or ''notapply'' for
do_apriori_sidelobe_mask');
      end

      % ------------------------------------------------------------------------
      % Select the psc and psp
      % ------------------------------------------------------------------------

      for g = 1:Ngrid_az
        for h = 1:Ngrid_r

          psc_phase_grid = repmat(NaN,Npsc_selections,Nifgs);
          psc_h2ph_grid = repmat(NaN,Npsc_selections,Nifgs);
          psc_amp_grid = repmat(NaN,Npsc_selections,Nslc);
          psc_az_grid = repmat(NaN,Npsc_selections,1);
          psc_r_grid = repmat(NaN,Npsc_selections,1);
          psc_amp_disp_grid = repmat(NaN,Npsc_selections,1);
          Npsc_grid = zeros(Npsc_selections,1);
          Npsp_grid = 0;

          temp = amp_disp(grid_array_az(index_az(g),1)-
begin_buffer_az+1:grid_array_az(index_az(g),2)-
begin_buffer_az+1,grid_array_r(index_r(h),1)-
begin_buffer_r+1:grid_array_r(index_r(h),2)-begin_buffer_r+1);

          if ~isempty(find(~isnan(temp(:)))) %only NaN (water)

            if (ref_cn(1)>=grid_array_az(index_az(g),1)&...
                ref_cn(1)<=grid_array_az(index_az(g),2)&...
                ref_cn(2)>=grid_array_r(index_r(h),1)&...
                ref_cn(2)<=grid_array_r(index_r(h),2))
              % inclusion of reference point in slc selection

              p = ref_cn(1)-grid_array_az(index_az(g),1)+1;
              q = ref_cn(2)-grid_array_r(index_r(h),1)+1;

              for z = 1:Npsc_selections
                psc_phase_grid(z,:) = ifgs_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
```

```
                psc_h2ph_grid(z,:) = h2ph_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
                psc_amp_grid(z,:) = amp_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
                psc_az_grid(z) = grid_array_az(index_az(g),1) + p - 1;
                psc_r_grid(z) = grid_array_r(index_r(h),1) + q - 1;
                psc_amp_disp_grid(z) = temp_sort(z);
                Npsc_grid(z) = Npsc_grid(z)+1;
              end

            else

              temp_sort = sort(temp(:));

              switch psc_selection_method
                case 'threshold'
                  for z = 1:Npsc_selections
                    if (temp_sort(z) < psc_threshold)
                      [p,q] = find(temp == temp_sort(z));
                      psc_phase_grid(z,:) =
ifgs_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
                      psc_h2ph_grid(z,:) =
h2ph_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
                      psc_amp_grid(z,:) =
amp_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
                      psc_az_grid(z) = grid_array_az(index_az(g),1) + p - 1;
                      psc_r_grid(z) = grid_array_r(index_r(h),1) + q - 1;
                      psc_amp_disp_grid(z) = temp_sort(z);
                      Npsc_grid(z) = Npsc_grid(z)+1;
                    end
                  end

                case 'eachgrid'
                  for z = 1:Npsc_selections
                    [p,q] = find(temp == temp_sort(z));
                    psc_phase_grid(z,:) =
ifgs_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
                    psc_h2ph_grid(z,:) =
h2ph_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
                    psc_amp_grid(z,:) = amp_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
                    psc_az_grid(z) = grid_array_az(index_az(g),1) + p - 1;
                    psc_r_grid(z) = grid_array_r(index_r(h),1) + q - 1;
                    psc_amp_disp_grid(z) = temp_sort(z);
                    Npsc_grid(z) = Npsc_grid(z)+1;
                  end

                case {'coherence','coherence_eachgrid'}

                  z = 1;
                  p_grid = repmat(NaN,Npsc_selections,1);
                  q_grid = repmat(NaN,Npsc_selections,1);
                  while (temp_sort(z)<psc_threshold) & z<=Npsc_selections
                    [p_grid(z),q_grid(z)] = find(temp == temp_sort(z));
                    psc_phase_grid(z,:) =
ifgs_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p_grid(z),grid_array_r(index_r(h),1)-
begin_buffer_r+q_grid(z),:);
                    psc_h2ph_grid(z,:) =
```

```
h2ph_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p_grid(z),grid_array_r(index_r(h),1)-
begin_buffer_r+q_grid(z),:);
                    psc_amp_grid(z,:) = amp_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p_grid(z),grid_array_r(index_r(h),1)-
begin_buffer_r+q_grid(z),:);
                    psc_az_grid(z) = grid_array_az(index_az(g),1) + p_grid(z) -
1;
                    psc_r_grid(z) = grid_array_r(index_r(h),1) + q_grid(z) - 1;
                    psc_amp_disp_grid(z) = temp_sort(z);
                    Npsc_grid(z) = Npsc_grid(z)+1;

                    z = z+1;
                  end

                  if z<=Npsc_selections

                    [temp_p temp_q] = find(temp<psp_threshold1);
                    Ntemp_p = size(temp_p,1);

                    if ~isempty(temp_p)
                      dist_gamma = repmat(NaN,Ntemp_p);
                      cpx_ifgs = repmat(NaN,Ntemp_p,Nifgs);
                      ifgs_h2ph = repmat(NaN,Ntemp_p,Nifgs);

                      %loop to read the ph and h2ph of all selected ps
                      for n_gamma = 1:Ntemp_p
                        %distance to the current point
                        dist_gamma(n_gamma,:) =  sqrt(((temp_p-
temp_p(n_gamma))*az_spacing) .^2 +...
                                                     ((temp_q-
temp_q(n_gamma))*r_spacing) .^2);

                        cpx_ifgs(n_gamma,:) =
cpx_ifgs_array(grid_array_az(index_az(g),1)-begin_buffer_az+temp_p(n_gamma),...

grid_array_r(index_r(h),1)-begin_buffer_r+temp_q(n_gamma),:);

                        ifgs_h2ph(n_gamma,:) =
h2ph_array(grid_array_az(index_az(g),1)-begin_buffer_az+temp_p(n_gamma),...

grid_array_r(index_r(h),1)-begin_buffer_r+temp_q(n_gamma),:);

                      end%end for n_gamma=1:size(temp_p,1)

                        diff_max=inf;
                        iter=1;

                        final_gamma = repmat(1,1,Ntemp_p);

                        while diff_max>0.005 && iter < 6
                          % ind_in = find( dist_gamma(n_gamma,:)<
psc_selection_gridsize  );
                          prev_gamma=final_gamma;

                          for n_gamma=1:Ntemp_p
                            [final_gamma(n_gamma) phase_res_dem_err] =
ps_gamma_estimation(cpx_ifgs,dist_gamma(n_gamma,:)', ...

ifgs_h2ph(n_gamma,:),prev_gamma');
                          end

                          [max_current_gamma in_max_current_gamma] =
max(final_gamma);
```

```
                          diff_max=abs(max_current_gamma-
prev_gamma(in_max_current_gamma));
                          iter=iter+1;

                        end%end while

                          %maximum of gammas
                          [sorted_gamma
ind_sorted_gamma]=sort(final_gamma,'descend');

                          z2 = 1;
                          while sorted_gamma(z2)>gamma_threshold &
z<=Npsc_selections
                            p=temp_p(ind_sorted_gamma(z2));
                            q=temp_q(ind_sorted_gamma(z2));
                            if isempty(find(ismember([p_grid(1:z) q_grid(1:z)],
[p q],'rows')))
                              p_grid(z) = p;
                              q_grid(z) = q;
                              psc_phase_grid(z,:) =
ifgs_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
                              psc_h2ph_grid(z,:) =
h2ph_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
                              psc_amp_grid(z,:) =
amp_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
                              psc_az_grid(z) = grid_array_az(index_az(g),1) + p
- 1;
                              psc_r_grid(z) = grid_array_r(index_r(h),1) + q -
1;
                              psc_amp_disp_grid(z) = temp(p,q);
                              Npsc_grid(z) = Npsc_grid(z)+1;
                              z = z+1;
                            end
                            z2 = z2+1;
                          end

                          if strcmp(psc_selection_method,'coherence_eachgrid')
%this else is from the if checking the threshold in coherence

                            disp('No good PSC found based in Spatial coherence
after calculations were done ');
                            [dummy ind_sorted] = sort((1-
final_gamma)'.*diag(temp(temp_p,temp_q)));

                            z2 = 1;
                            while z<=Npsc_selections
                              p = temp_p(ind_sorted(z2));
                              q = temp_q(ind_sorted(z2));
                              if isempty(find(ismember([p_grid(1:z)
q_grid(1:z)],[p q],'rows')))
                                p_grid(z) = p;
                                q_grid(z) = q;
                                psc_phase_grid(z,:) =
ifgs_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
                                psc_h2ph_grid(z,:) =
h2ph_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
                                psc_amp_grid(z,:) =
amp_array(grid_array_az(index_az(g),1)-
```

```
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
                                psc_az_grid(z) = grid_array_az(index_az(g),1) +
p - 1;
                                psc_r_grid(z) = grid_array_r(index_r(h),1) + q -
1;
                                psc_amp_disp_grid(z) = temp_sort(z);
                                Npsc_grid(z) = Npsc_grid(z)+1;
                                z = z+1;
                              end
                              z2 = z2+1;
                            end %end while
                          end %end if
strcmp(psc_selection_method,'coherence_eachgrid')

                            clear final_gamma  sorted_gamma ind_sorted_gamma

                    elseif strcmp(psc_selection_method,'coherence_eachgrid')
%Else we just try with the best PS

                      z2 = 1;
                      while z<=Npsc_selections
                        [p,q] = find(temp == temp_sort(z2));
                        if isempty(find(ismember([p_grid(1:z) q_grid(1:z)],[p
q],'rows')))
                          p_grid(z) = p;
                          q_grid(z) = q;
                          psc_phase_grid(z,:) =
ifgs_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
                          psc_h2ph_grid(z,:) =
h2ph_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
                          psc_amp_grid(z,:) =
amp_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p,grid_array_r(index_r(h),1)-begin_buffer_r+q,:);
                          psc_az_grid(z) = grid_array_az(index_az(g),1) + p - 1;
                          psc_r_grid(z) = grid_array_r(index_r(h),1) + q - 1;
                          psc_amp_disp_grid(z) = temp_sort(z);
                          Npsc_grid(z) = Npsc_grid(z)+1;
                          z = z+1;
                        end
                        z2 = z2+1;
                      end

                    end %end if ~isempty(temp_p)

                  end % end if z<=Npsc_selections

                otherwise
                  error('Error: the psc selection method was not specified
correctly');
              end

            end

            switch ps_eval_method
              case 'psp'

                Nel_grid = size(temp,1)*size(temp,2);

                if ischar(ps_aoi)
                  aoi_mask_buffer = aoi_mask(grid_array_az(index_az(g),1)-
begin_buffer_az+1:grid_array_az(index_az(g),2)-
begin_buffer_az+1,grid_array_r(index_r(h),1)-
```

```
begin_buffer_r+1:grid_array_r(index_r(h),2)-begin_buffer_r+1,:);
                  [aoi_index1,aoi_index2] = find(aoi_mask_buffer==1);
                  Npsp_grid = length(aoi_index1);
                  psp_az_grid = repmat(NaN,Npsp_grid,1);
                  psp_r_grid = repmat(NaN,Npsp_grid,1);
                  psp_phase_grid = repmat(NaN,Npsp_grid,Nifgs);
                  psp_h2ph_grid = repmat(NaN,Npsp_grid,Nifgs);
                  psp_amp_grid = repmat(NaN,Npsp_grid,Nslc);
                  psp_amp_disp_grid = repmat(NaN,Npsp_grid,1);
                  for m = 1:Npsp_grid
                    psp_az_grid(m) = grid_array_az(index_az(g),1) +
aoi_index1(m)-1;
                    psp_r_grid(m) = grid_array_r(index_r(h),1) + aoi_index2(m)-
1;
                    psp_phase_grid(m,:) =
ifgs_array(grid_array_az(index_az(g),1)-
begin_buffer_az+aoi_index1(m),grid_array_r(index_r(h),1)-
begin_buffer_r+aoi_index2(m),:);
                    psp_h2ph_grid(m,:) =
h2ph_array(grid_array_az(index_az(g),1)-
begin_buffer_az+aoi_index1(m),grid_array_r(index_r(h),1)-
begin_buffer_r+aoi_index2(m),:);
                    psp_amp_grid(m,:) = amp_array(grid_array_az(index_az(g),1)-
begin_buffer_az+aoi_index1(m),grid_array_r(index_r(h),1)-
begin_buffer_r+aoi_index2(m),:);
                  end
                  overlap = 0;
                elseif ~isempty(ps_aoi)
                  overlap = (max(0, min(ps_aoi(2)-
crop(1)+1,grid_array_az(index_az(g),2)) - max(ps_aoi(1)-
crop(1)+1,grid_array_az(index_az(g),1))))*(max(0, min(ps_aoi(4)-
crop(3)+1,grid_array_r(index_r(h),2)) - max(ps_aoi(3)-
crop(3)+1,grid_array_r(index_r(h),1))));
                  aoi_mask_buffer = zeros(size(temp));
                else
                  overlap = 0;
                  aoi_mask_buffer = zeros(size(temp));
                end

                if overlap ~= 0
                  psp_az_grid =
reshape(repmat((grid_array_az(index_az(g),1):grid_array_az(index_az(g),2))',size
(temp,2),1),Nel_grid,1);
                  psp_r_grid =
reshape(repmat(grid_array_r(index_r(h),1):grid_array_r(index_r(h),2),size(temp,1
),1),Nel_grid,1);
                  psp_phase_grid =
reshape(ifgs_array(grid_array_az(index_az(g),1)-
begin_buffer_az+1:grid_array_az(index_az(g),2)-
begin_buffer_az+1,grid_array_r(index_r(h),1)-
begin_buffer_r+1:grid_array_r(index_r(h),2)-begin_buffer_r+1,:),Nel_grid,Nifgs);
                  psp_h2ph_grid =
reshape(h2ph_array(grid_array_az(index_az(g),1)-
begin_buffer_az+1:grid_array_az(index_az(g),2)-
begin_buffer_az+1,grid_array_r(index_r(h),1)-
begin_buffer_r+1:grid_array_r(index_r(h),2)-begin_buffer_r+1,:),Nel_grid,Nifgs);
                  psp_amp_grid = reshape(amp_array(grid_array_az(index_az(g),1)-
begin_buffer_az+1:grid_array_az(index_az(g),2)-
begin_buffer_az+1,grid_array_r(index_r(h),1)-
begin_buffer_r+1:grid_array_r(index_r(h),2)-begin_buffer_r+1,:),Nel_grid,Nifgs);

                  sl_mask_grid = reshape(sl_mask(grid_array_az(index_az(g),1)-
begin_buffer_az+1:grid_array_az(index_az(g),2)-
begin_buffer_az+1,grid_array_r(index_r(h),1)-
```

```
begin_buffer_r+1:grid_array_r(index_r(h),2)-begin_buffer_r+1),Nel_grid,1);

                  if ~isempty(find(sl_mask_grid==1))
                    psp_az_grid(sl_mask_grid) = [];
                    psp_r_grid(sl_mask_grid) = [];
                    psp_phase_grid(sl_mask_grid,:) = [];
                    psp_h2ph_grid(sl_mask_grid,:) = [];
                    psp_amp_grid(sl_mask_grid,:) = [];
                  end

                  Npsp_grid = length(psp_az_grid);

                else

                  switch psp_selection_method
                    case 'highamp'

                      if ischar(ps_aoi)&&Npsp_grid~=0
                        psp_az_grid = [psp_az_grid;repmat(NaN,Nel_grid,1)];
                        psp_r_grid = [psp_r_grid;repmat(NaN,Nel_grid,1)];
                        psp_phase_grid =
[psp_phase_grid;repmat(NaN,Nel_grid,Nifgs)];
                        psp_h2ph_grid =
[psp_h2ph_grid;repmat(NaN,Nel_grid,Nifgs)];
                        psp_amp_grid =
[psp_h2ph_grid;repmat(NaN,Nel_grid,Nslc)];
                      else
                        Npsp_grid = 0;
                        psp_az_grid = repmat(NaN,Nel_grid,1);
                        psp_r_grid = repmat(NaN,Nel_grid,1);
                        psp_phase_grid = repmat(NaN,Nel_grid,Nifgs);
                        psp_h2ph_grid = repmat(NaN,Nel_grid,Nifgs);
                        psp_amp_grid = repmat(NaN,Nel_grid,Nslc);
                      end

                      for m = 1:size(temp,1)
                        for n = 1:size(temp,2)
                          [dummy,index_psp] =
find(amp_array(grid_array_az(index_az(g),1)-
begin_buffer_az+m,grid_array_r(index_r(h),1)-begin_buffer_r+n,:)>psp_amplitude);
                          if
(length(index_psp)>psp_threshold1*Nifgs)&&(aoi_mask_buffer(m,n)==0)
                            Npsp_grid = Npsp_grid + 1;
                            psp_az_grid(Npsp_grid) =
grid_array_az(index_az(g),1) + m-1;
                            psp_r_grid(Npsp_grid) = grid_array_r(index_r(h),1) +
n-1;
                            psp_phase_grid(Npsp_grid,:) =
ifgs_array(grid_array_az(index_az(g),1)-
begin_buffer_az+m,grid_array_r(index_r(h),1)-begin_buffer_r+n,:);
                            psp_h2ph_grid(Npsp_grid,:) =
h2ph_array(grid_array_az(index_az(g),1)-
begin_buffer_az+m,grid_array_r(index_r(h),1)-begin_buffer_r+n,:);
                            psp_amp_grid(Npsp_grid,:) =
h2ph_amp(grid_array_az(index_az(g),1)-
begin_buffer_az+m,grid_array_r(index_r(h),1)-begin_buffer_r+n,:);
                          end
                        end
                      end
                      psp_az_grid(Npsp_grid+1:end) = [];
                      psp_r_grid(Npsp_grid+1:end) = [];
                      psp_phase_grid(Npsp_grid+1:end,:) = [];
                      psp_h2ph_grid(Npsp_grid+1:end,:) = [];
                      psp_amp_grid(Npsp_grid+1:end,:) = [];
```

```
                    case 'ampdisp'

                      [p,q] = find((temp <
psp_threshold1)&(aoi_mask_buffer==0));

                      if ischar(ps_aoi)&&Npsp_grid~=0
                        psp_az_grid = [psp_az_grid;grid_array_az(index_az(g),1)
+ p - 1];
                        psp_r_grid = [psp_r_grid;grid_array_r(index_r(h),1) + q
- 1];
                        psp_phase_grid =
[psp_phase_grid;repmat(NaN,length(p),Nifgs)];
                        psp_h2ph_grid =
[psp_h2ph_grid;repmat(NaN,length(p),Nifgs)];
                        psp_amp_grid =
[psp_amp_grid;repmat(NaN,length(p),Nslc)];
                        psp_amp_disp_grid =
[psp_amp_disp_grid;repmat(NaN,length(p),1)];
                      else
                        psp_az_grid = grid_array_az(index_az(g),1) + p - 1;
                        psp_r_grid = grid_array_r(index_r(h),1) + q - 1;
                        psp_phase_grid = repmat(NaN,length(p),Nifgs);
                        psp_h2ph_grid = repmat(NaN,length(p),Nifgs);
                        psp_amp_grid = repmat(NaN,length(p),Nslc);
                        psp_amp_disp_grid = repmat(NaN,length(p),1);
                        Npsp_grid = 0;
                      end

                      for z = 1:length(p)
                        Npsp_grid = Npsp_grid + 1;
                        psp_phase_grid(Npsp_grid,:) =
ifgs_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p(z),grid_array_r(index_r(h),1)-begin_buffer_r+q(z),:);
                        psp_h2ph_grid(Npsp_grid,:) =
h2ph_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p(z),grid_array_r(index_r(h),1)-begin_buffer_r+q(z),:);
                        psp_amp_grid(Npsp_grid,:) =
amp_array(grid_array_az(index_az(g),1)-
begin_buffer_az+p(z),grid_array_r(index_r(h),1)-begin_buffer_r+q(z),:);
                        psp_amp_disp_grid(Npsp_grid) = temp(p(z),q(z));
                      end

                    otherwise
                      error('You specified a wrong psp_selection_method.');
                  end %end switch
                end %end if overlap ~= 0
              end %end switch ps_eval_method

              %
-------------------------------------------------------------------
              % Write the data to file
              %
-------------------------------------------------------------------

              if max(Npsc_grid) ~= 0
                for z = 1:Npsc_selections
                  psc_total = [index_az(g) index_r(h) psc_az_grid(z) psc_r_grid(z)
psc_phase_grid(z,:) psc_h2ph_grid(z,:) psc_amp_grid(z,:) psc_amp_disp_grid(z)];
                  fwrite(psc_temp_fid(z),psc_total','double');
                end
                Npsc = Npsc + Npsc_grid;
              end
```

```
            switch ps_eval_method
              case 'psp'
                if Npsp_grid ~= 0
                  psp_total = [repmat(index_az(g),Npsp_grid,1)
repmat(index_r(h),Npsp_grid,1) ...
                    psp_az_grid psp_r_grid psp_phase_grid psp_h2ph_grid];
                  for z = 1:Npsc_selections
                    fwrite(psp_fid(z),psp_total','double');
                    % exact copies, but I want to avoid unix('cp ..')
                    % commands to enable Windows use
                  end

                  psp_validation = [psp_amp_disp_grid psp_amp_grid];
                  fwrite(psp_validation_fid,psp_validation','double');

                  Npsp = Npsp + repmat(Npsp_grid,Npsc_selections,1);
                  clear psp_az_buffer psp_r_buffer psp_phase_buffer
psp_h2ph_buffer
                end
            end

          end %if only NaN (water)

        end %for
      end %for

    else
      display('Only water, no psc/psp selected for this buffer.');
    end

    % ---------------------------------------------------------------------
    % Write the data to file
    % ---------------------------------------------------------------------

    if w == Nbuffers_r % complete in range direction

      %      switch orbit
      %       case 'asc'
      %        multi_refl_map = log10(multi_refl_map);
      %       case 'desc'
      %        multi_refl_map = log10(flipud(fliplr(multi_refl_map)));
      %       otherwise
      %        error('The orbit (asc/desc) is not specified correctly');
      %      end

      multi_refl_map = log10(multi_refl_map);
      fwrite(mrm_fid,multi_refl_map','single'); % transpose is important!
      fwrite(amp_disp_fid,amp_disp_map','single');
      fwrite(side_lobe_fid,side_lobe_mask','uint8');

      if save_livetime
          fwrite(livetime_fid,livetime','uint8');
      end

    end

  end %for

end % end main loop



% ---------------------------------------------------------------------
% Close all open files
```

```
% ------------------------------------------------------------------------

fclose('all');
clear ifgs_array slc_array h2ph_array amp_array multi_refl_map amp_disp_map



% ------------------------------------------------------------------------
% Read psc's from file
% ------------------------------------------------------------------------

max_Npsc = max(Npsc);
psc_grid_az = repmat(NaN,max_Npsc,Npsc_selections);
psc_grid_r = repmat(NaN,max_Npsc,Npsc_selections);
psc_az = repmat(NaN,max_Npsc,Npsc_selections);
psc_r = repmat(NaN,max_Npsc,Npsc_selections);
psc_phase = repmat(NaN,[max_Npsc Nifgs Npsc_selections]);
psc_h2ph = repmat(NaN,[max_Npsc Nifgs Npsc_selections]);
psc_amp = repmat(NaN,[max_Npsc Nslc Npsc_selections]);
psc_amp_disp = repmat(NaN,max_Npsc,Npsc_selections);


for z = 1:Npsc_selections
  psc_temp_fid = fopen([project_id '_psc_temp_sel' num2str(z) '.raw'],'r'); %
file with temporary psc info
  psc_data = fread(psc_temp_fid,[2*Nifgs+Nslc+5 max_Npsc],'double')';
  % the transpose at the end is obviously very important...
  fclose(psc_temp_fid);

  psc_grid_az(:,z) = psc_data(:,1);
  psc_grid_r(:,z) = psc_data(:,2);
  psc_az(:,z) = psc_data(:,3);
  psc_r(:,z) = psc_data(:,4);
  psc_phase(:,:,z) = psc_data(:,5:Nifgs+4);
  psc_h2ph(:,:,z) = psc_data(:,Nifgs+5:2*Nifgs+4);
  psc_amp(:,:,z) = psc_data(:,2*Nifgs+5:2*Nifgs+Nslc+4);
  psc_amp_disp(:,z) = psc_data(:,2*Nifgs+Nslc+5);
end


switch psc_distribution
  case 'uniform'
    grid_array_az_shift = grid_array_az(:,1:2);
    grid_array_az_shift(2:end,1) = grid_array_az(2:end,1)- ...

repmat(floor(Nlines_g/2),size(grid_array_az,1)-1,1);
    grid_array_az_shift(1:end,2) = grid_array_az(1:end,2)- ...

repmat(floor(Nlines_g/2),size(grid_array_az,1),1);
    grid_array_az_shift(end+1,:) = [Nlines-floor(Nlines_g/2)+1 ...
      Nlines];

    for z = 1:Npsc_selections
      count = 0;
      psc_new = repmat(NaN,max_Npsc,1);
      for v = 1:size(grid_array_az_shift,1)
        for w = 1:size(grid_array_r)
          index =
find((psc_az(:,z)>=grid_array_az_shift(v,1))&(psc_az(:,z)<= ...

grid_array_az_shift(v,2))&(psc_r(:,z)>=grid_array_r(w,1))&(psc_r(:,z)<=grid_arra
y_r(w,2)));
          if ~isempty(index)
            count = count + 1;
```

```
            ref_index = find(ismember([psc_az(index,z)
psc_r(index,z)],ref_cn,'rows'));
            %keep ref point
            if isempty(ref_index)
              [dummy,index2] = min(psc_amp_disp(index,z));
              psc_new(count) = index(index2);
            else
              psc_new(count) = index(ref_index);
            end

          end
        end
      end

      Npsc(z) = count;
      psc_new(Npsc(z)+1:end) = [];

      psc_grid_az(:,z) = [psc_grid_az(psc_new,z); repmat(NaN,max_Npsc-
Npsc(z),1)];
      psc_grid_r(:,z) = [psc_grid_r(psc_new,z); repmat(NaN,max_Npsc-Npsc(z),1)];
      psc_az(:,z) = [psc_az(psc_new,z); repmat(NaN,max_Npsc-Npsc(z),1)];
      psc_r(:,z) = [psc_r(psc_new,z); repmat(NaN,max_Npsc-Npsc(z),1)];
      psc_phase(:,:,z) = [psc_phase(psc_new,:,z); repmat(NaN,max_Npsc-
Npsc(z),Nifgs)];
      psc_h2ph(:,:,z) = [psc_h2ph(psc_new,:,z); repmat(NaN,max_Npsc-
Npsc(z),Nifgs)];
      psc_amp(:,:,z) = [psc_amp(psc_new,:,z); repmat(NaN,max_Npsc-
Npsc(z),Nslc)];
      psc_amp_disp(:,z) = [psc_amp_disp(psc_new,z); repmat(NaN,max_Npsc-
Npsc(z),1)];
    end

    grid_array_r_shift = grid_array_r(:,1:2);
    grid_array_r_shift(2:end,1) = grid_array_r(2:end,1)- ...

repmat(floor(Npixels_g/2),size(grid_array_r,1)-1,1);
    grid_array_r_shift(1:end,2) = grid_array_r(1:end,2)- ...

repmat(floor(Npixels_g/2),size(grid_array_r,1),1);
    grid_array_r_shift(end+1,:) = [Npixels-floor(Npixels_g/2)+1 ...
      Npixels];

    for z = 1:Npsc_selections
      count = 0;
      psc_new = repmat(NaN,max_Npsc,1);
      for v = 1:size(grid_array_az_shift,1)
        for w = 1:size(grid_array_r_shift)
          index =
find((psc_az(:,z)>=grid_array_az_shift(v,1))&(psc_az(:,z)<= ...

grid_array_az_shift(v,2))&(psc_r(:,z)>=grid_array_r_shift(w,1))&(psc_r(:,z)<=gri
d_array_r_shift(w,2)));
          if ~isempty(index)
            count = count + 1;
            ref_index = find(ismember([psc_az(index,z)
psc_r(index,z)],ref_cn,'rows'));
            %keep ref point
            if isempty(ref_index)
              [dummy,index2] = min(psc_amp_disp(index,z));
              psc_new(count) = index(index2);
            else
              psc_new(count) = index(ref_index);
            end
```

```
          end
        end
      end

      Npsc(z) = count;
      psc_new(Npsc(z)+1:end) = [];

      psc_grid_az(:,z) = [psc_grid_az(psc_new,z); repmat(NaN,max_Npsc-
Npsc(z),1)];
      psc_grid_r(:,z) = [psc_grid_r(psc_new,z); repmat(NaN,max_Npsc-Npsc(z),1)];
      psc_az(:,z) = [psc_az(psc_new,z); repmat(NaN,max_Npsc-Npsc(z),1)];
      psc_r(:,z) = [psc_r(psc_new,z); repmat(NaN,max_Npsc-Npsc(z),1)];
      psc_phase(:,:,z) = [psc_phase(psc_new,:,z); repmat(NaN,max_Npsc-
Npsc(z),Nifgs)];
      psc_h2ph(:,:,z) = [psc_h2ph(psc_new,:,z); repmat(NaN,max_Npsc-
Npsc(z),Nifgs)];
      psc_amp(:,:,z) = [psc_amp(psc_new,:,z); repmat(NaN,max_Npsc-
Npsc(z),Nslc)];
      psc_amp_disp(:,z) = [psc_amp_disp(psc_new,z); repmat(NaN,max_Npsc-
Npsc(z),1)];
    end

    max_Npsc = max(Npsc);
    psc_grid_az = psc_grid_az(1:max_Npsc,:);
    psc_grid_r = psc_grid_r(1:max_Npsc,:);
    psc_az = psc_az(1:max_Npsc,:);
    psc_r = psc_r(1:max_Npsc,:);
    psc_phase = psc_phase(1:max_Npsc,:,:);
    psc_h2ph = psc_h2ph(1:max_Npsc,:,:);
    psc_amp = psc_amp(1:max_Npsc,:,:);
    psc_amp_disp = psc_amp_disp(1:max_Npsc,:);

  case 'nonuniform'
    %do nothing
  otherwise
    error(['You should either specify ''uniform'' or ''nonuniform'' for' ...
      ' the ''psc_distribution''']);
end %end switch


% ---------------------------------------------------------------------
% Sort the final psc's on coordinates
% ---------------------------------------------------------------------

for z = 1:Npsc_selections
  [dummy,sort_index] = sortrows([psc_az(:,z) psc_r(:,z)]);
  psc_grid_az(:,z) = psc_grid_az(sort_index,z);
  psc_grid_r(:,z) = psc_grid_r(sort_index,z);
  psc_az(:,z) = dummy(:,1);
  psc_r(:,z) = dummy(:,2);
  psc_phase(:,:,z) = psc_phase(sort_index,:,z);
  psc_h2ph(:,:,z) = psc_h2ph(sort_index,:,z);
  psc_amp(:,:,z) = psc_amp(sort_index,:,z);
  psc_amp_disp(:,z) = psc_amp_disp(sort_index,z);
end


% ---------------------------------------------------------------------
% Make plots of amplitudes
% ---------------------------------------------------------------------

if strcmp(detail_plots,'y')
  Btemp_slc = [Btemp(Btemp<0)' 0 Btemp(Btemp>0)'];
  for z = 1:Npsc_selections
```

```
    for w = 1:Npsc(z)
      figure(fig+1);
      set(gcf,'visible','off');hold on;
      plot(Btemp_slc,psc_amp(w,:,z),'b*');
      axis([min(Btemp_slc)-0.5 max(Btemp_slc)+0.5 0 1000]);
      plot(Btemp_slc(slc_selection),psc_amp(w,slc_selection,z),'r*');
      mean_check = mean(psc_amp(w,slc_selection,z));
      std_check = std(psc_amp(w,slc_selection,z),0);
      amp_disp_check = std_check/mean_check;
      plot(Btemp_slc,repmat(mean_check,1,Nslc),'r');
      plot(Btemp_slc,repmat(mean_check+std_check,1,Nslc),'r--');
      plot(Btemp_slc,repmat(mean_check-std_check,1,Nslc),'r--');
      plot(Btemp_slc,repmat(mean_check+(mean_check/4),1,Nslc),'k--');
      plot(Btemp_slc,repmat(mean_check-(mean_check/4),1,Nslc),'k--');
      xlim = get(gca,'xlim');
      xmin = min(xlim);
      xmax = max(xlim);
      ylim = get(gca,'ylim');
      ymin = min(ylim);
      ymax = max(ylim);
      text(xmax-(xmax-xmin)/3,ymax-(ymax-ymin)/30,['amp_disp = '
num2str(amp_disp_check)],'interpreter','none');
      text(xmax-(xmax-xmin)/3,ymax-2*(ymax-ymin)/30,['threshold = '
num2str(psc_threshold)],'interpreter','none');
      xlabel('Btemp [y]');
      ylabel('Amplitude [-]');
      hold off;
      print('-dpng',['plots/' project_id '_psc_amp_sel' num2str(z) '_'
num2str(w) '.png']);
      close(fig+1);
    end
  end
end

% --------------------------------------------------------------------
% Write final psc's to file
% --------------------------------------------------------------------

for z = 1:Npsc_selections
  psc_az_selection = psc_az(1:Npsc(z),z);
  psc_r_selection = psc_r(1:Npsc(z),z);
  psc_array = [(1:Npsc(z))' ones(Npsc(z),1)];

  psc_total = [psc_array psc_grid_az(1:Npsc(z),z) psc_grid_r(1:Npsc(z),z) ...
               psc_az_selection psc_r_selection psc_phase(1:Npsc(z),:,z) ...
               psc_h2ph(1:Npsc(z),:,z) psc_amp_disp(1:Npsc(z),z)];
  psc_fid = fopen([project_id '_psc_2orig_sel' num2str(z) '.raw'],'w'); % file
with psc info
  fwrite(psc_fid,psc_total','double');
  fclose(psc_fid);

  psc_validation_fid = fopen([project_id '_psc_validation_sel' ...
                      num2str(z) '.raw'],'w');
  fwrite(psc_validation_fid,psc_amp(1:Npsc(z),:,z)','double');
  fclose(psc_validation_fid);


  save([project_id '_selected_psc_sel' num2str(z)
'.mat'],'psc_r_selection','psc_az_selection','grid_array_az','grid_array_r');

end

clear psc_total psc_phase psc_h2ph psc_amp
```

```
% ------------------------------------------------------------------------
% Save mrm as image
% ------------------------------------------------------------------------

Nlines_buffer = min(Nlines,floor(max_mem_buffer/(8*Npixels)));
Nlines_buffer_new = ceil(Nlines_buffer);
Nbuffer = ceil(Nlines/Nlines_buffer);
Nlines_rem_buffer = rem(Nlines,Nlines_buffer);

mrm_fid_in = fopen([project_id '_mrm.raw'],'r');
mrm_fid_out = fopen([project_id '_mrm_uint8.raw'],'w');
for v = 1:Nbuffer
  sl = (v-1)*Nlines_buffer_new+1;
  if (v==Nbuffer)&&(Nlines_rem_buffer~=0)
    Nlines_buffer = Nlines_rem_buffer;
    el = Nlines;
  else
    el = v*Nlines_buffer_new;
  end

  mrm = fread(mrm_fid_in,[Npixels Nlines_buffer],'*single')';

  mrm_min = min(mrm(:));
  mrm_max = max(mrm(:));
  mrm_new = uint8(255*(mrm-mrm_min)./(mrm_max-mrm_min));
  %mrm_new = imadjust(mrm_new); %stretch,
  % switched off, caused blocks in image, FvL 19-01-2010

  fwrite(mrm_fid_out,mrm_new','uint8');
end
fclose(mrm_fid_in);
fclose(mrm_fid_out);


% ------------------------------------------------------------------------
% Output
% ------------------------------------------------------------------------

fid_res = fopen([project_id '_resfile.txt'],'a');
for z = 1:Npsc_selections
  fprintf(1,'In total, %4.0f PSC have been selected in selection
%2.0f\n',Npsc(z),z);

  fprintf(fid_res,[datestr(now) ', number of PSCs in selection %g:
%g\n'],z,Npsc(z));
end
fprintf(1,'In total, %4.0f PSP have been selected\n',Npsp);
fprintf(fid_res,[datestr(now) ', number of PSP selected: %g\n'],Npsp);
fclose('all');
```